

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Título del Proyecto:

Sistema Inteligente de Seguridad y Gestión de Tráfico Urbano: Cimentación y Monitoreo Local FASE2

PONDERACIÓN: 25

Horas Aproximadas: 22

Índice

1. Resumen Ejecutivo	3
2. Competencia que desarrollaremos	3
3. Objetivos del Aprendizaje	4
3.1 Objetivo General.....	4
3.2 Objetivos Específicos	4
4. Enunciado del Proyecto	5
4.1 Descripción del problema a resolver.....	5
4.2 Alcance del proyecto	5
4.3 Entregables	10
5. Metodología	10
6. Desarrollo de Habilidades Blandas.....	11
6.1 Proyectos en Grupo.....	12
7. Cronograma	12
8. Rúbrica de Calificación	13
8.1 Requisitos para optar a la calificación.....	13
8.2 Valores	14
8.3 Comentarios Generales.....	14

1. Resumen Ejecutivo

Este proyecto aborda los desafíos de gestión del tráfico urbano y seguridad ciudadana en zonas históricas de alta densidad, como el Centro Histórico de la Zona 1 de Guatemala, donde la movilidad ineficiente y la falta de vigilancia proactiva generan riesgos para la población y el patrimonio.

El sistema propuesto es una solución integrada basada en Internet de las Cosas (IoT) e Inteligencia Artificial (IA) que permite monitorear en tiempo real el flujo vehicular, detectar incendios, activar alertas de emergencia y reconocer comportamientos de riesgo, como vehículos que cruzan semáforos en rojo o personas en listas de vigilancia.

La solución se compone de sensores conectados a microcontroladores Arduino, que capturan datos ambientales y de tráfico. Estos se comunican mediante conexión serial con una Raspberry Pi, que actúa como nodo central de comunicación y publica la información vía MQTT. Las tareas de procesamiento de video (reconocimiento de placas y rostros) son realizadas por computadoras externas que ejecutan modelos de IA, enviando los resultados a la Raspberry Pi para su integración y almacenamiento.

Los datos son gestionados por una API y una base de datos, y son accesibles para los ciudadanos a través de una aplicación móvil desarrollada en Flutter, que muestra alertas, estado del tráfico y tiempo de llegada del transporte público. Además, se implementa un sistema de códigos QR en puntos históricos para fomentar el turismo mediante información digital.

En resumen, este proyecto desarrolla un sistema inteligente, escalable y conectado que mejora la seguridad, optimiza la movilidad y promueve la cultura, utilizando tecnología accesible y arquitecturas modernas de IoT y computación distribuida.

2. Competencia que desarrollaremos

- Integra soluciones IoT utilizando hardware, software y análisis de datos mediante metodologías activas como ABP (aprendizaje basado en proyectos) para resolver problemas prácticos en áreas como automatización y monitoreo ambiental.
- Configura un sistema IoT basado en Arduino y Raspberry Pi utilizando protocolos como MQTT y bibliotecas específicas para implementar comunicación segura entre dispositivos embebidos.
- Diseña una interfaz gráfica para visualización de datos en tiempo real usando herramientas como Processing o p5.js para representar información recolectada desde sensores conectados a Arduino.
- Implementa rutinas de interrupción en sistemas embebidos usando Arduino IDE y bibliotecas específicas para gestionar eventos asíncronos de sensores y actuadores en aplicaciones de automatización que requieren respuesta en tiempo real.
- Configura sistemas de comunicación IoT utilizando protocolos como HTTP o COAP mediante servidores API RESTful y dispositivos embebidos para optimizar la transmisión de datos en redes inalámbricas de bajo consumo.

3. Objetivos del Aprendizaje

3.1 Objetivo General

Diseñar e implementar un sistema inteligente de seguridad y gestión de tráfico urbano basado en IoT e Inteligencia Artificial, que integre sensores, microcontroladores y computadoras externas para monitorear condiciones ambientales, detectar infracciones y emergencias, y proporcionar información en tiempo real a través de una aplicación , utilizando una Raspberry Pi como nodo central de comunicación y coordinación, sin ejecutar modelos de IA directamente sobre ella.

3.2 Objetivos Específicos

- Diseñar y construir un prototipo a escala de un sistema de seguridad y gestión de tráfico, utilizando microcontroladores Arduino para el control local y la captura de datos de sensores.
- Implementar un framework de comunicación IoT basado en el protocolo MQTT para garantizar la transmisión de datos de manera eficiente y desacoplada entre los dispositivos de campo, el servidor y las aplicaciones cliente.
- Desarrollar una plataforma de software centralizada, que incluya una base de datos (SQL o NoSQL) para el almacenamiento histórico de eventos y una API para el consumo de datos.
- Crear una aplicación móvil multiplataforma (Flutter) que sirva como interfaz para el ciudadano, mostrando información en tiempo real del estado del tráfico, la ubicación simulada del transporte público y alertas de seguridad.
- Integrar un módulo de Inteligencia Artificial en computadoras externas, que procesarán video en tiempo real y enviarán los resultados a la Raspberry Pi, la cual actuará como nodo central de integración y comunicación.
- Fomentar el turismo y la cultura mediante la implementación de un sistema de códigos QR en puntos de interés histórico, vinculados a una página web informativa.

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

El presente proyecto propone el diseño, desarrollo e implementación de un sistema integral para la gestión inteligente del tráfico y la seguridad ciudadana en un entorno urbano simulado, basado en el Centro Histórico de la Zona 1 de Guatemala. Ante los crecientes desafíos en materia de movilidad y seguridad en áreas de alta densidad poblacional y valor patrimonial, se hace fundamental explorar soluciones tecnológicas innovadoras.

Este sistema se fundamenta en la tecnología de Internet de las Cosas (IoT) para la recopilación de datos en tiempo real a través de una red de sensores distribuidos. La información recolectada abarca desde el flujo vehicular, la detección de incendios, detección de sismo, hasta la activación de alertas de pánico. Posteriormente, los datos son procesados, almacenados y analizados para la toma de decisiones automatizada y la presentación de información relevante a los usuarios finales a través de una aplicación móvil.

La fase culminante del proyecto integra un componente de Inteligencia Artificial (IA) mediante el uso de una Raspberry Pi como nodo central de comunicación, mientras que las tareas de procesamiento de video se realizarán en computadoras externas que enviarán la información procesada a la Raspberry Pi. Esto permite mantener un sistema escalable y eficiente, adaptándose mejor a las limitaciones de hardware de la plataforma embebida. El sistema permitirá funcionalidades avanzadas como el reconocimiento de matrículas para la detección de infracciones, el reconocimiento facial para la identificación de individuos en listas de vigilancia, armas blancas, fortaleciendo así la seguridad proactiva en el área de operación.

4.2 Alcance del proyecto

Esta segunda fase representa la evolución fundamental del proyecto, transformando el sistema de monitoreo local desarrollado en la Fase 1 en una arquitectura de **Internet de las Cosas (IoT) robusta, conectada y persistente**. El objetivo principal es trascender la monitorización por cable para establecer un ecosistema de comunicación de datos en red, sentando las bases para el almacenamiento histórico y la interacción del usuario final a través de una aplicación móvil. En esta etapa, la Raspberry Pi se establece como el epicentro computacional, orquestando el flujo de información y conectando los dispositivos de campo con la plataforma de software.

Componentes Tecnológicos

HARDWARE:

- **Raspberry Pi:** Se establece como el **nodo central y orquestador de la red**. Actuará como un servidor dedicado, ejecutando no solo el *broker* MQTT y la API, sino también los *scripts* de servicio que gestionarán la lógica de comunicación y el procesamiento intermedio de datos.
- **Microcontrolador Arduino UNO:** Mantiene su rol crucial como la **interfaz directa con el mundo físico**. Continuará operando como el nodo de campo, encargado de la lectura en tiempo real de los sensores y la activación precisa de los actuadores (alarmas). Su comunicación con la Raspberry Pi se realizará a través de una conexión serial por cable USB, garantizando un enlace de datos fiable.

- **Pieza Impresa en 3D:** Se materializa la capacidad de prototipado rápido diseñando e imprimiendo una **pieza física a medida**. Esta se integrará de forma ergonómica y estética a la maqueta, mejorando su funcionalidad (ej. un soporte optimizado para sensores, una carcasa protectora para la electrónica) y demostrando la personalización del hardware para el entorno.
- **Estética:** debe agregar elementos a la maqueta para mejorar la estética de la misma.
- **Funcionalidades anteriores:** Las funcionalidades anteriores físicas se mantendrán en funcionamiento.

SOFTWARE:

- **Backend (Desplegado en Raspberry Pi):**
 - **Broker MQTT (ej. Mosquitto):** Se implementará como el sistema nervioso central de la comunicación para garantizar un intercambio de mensajes **asíncrono, desacoplado y de bajo consumo**.
 - **Base de Datos (SQL o NoSQL):** Se desplegará un sistema gestor (ej. PostgreSQL o MongoDB) para asegurar la **persistencia y la capacidad de análisis histórico** de cada evento y lectura del sistema, esta base de datos estará online.
 - **API RESTful:** Se desarrollará utilizando un framework moderno (ej. FastAPI, Express) para actuar como una **puerta de acceso segura y estandarizada a los datos**, desacoplando completamente la lógica del backend de la presentación en el frontend, esta api estará online.
- **Frontend (Aplicación Móvil):**
 - **Framework Flutter:** Se utilizará para construir una aplicación móvil multiplataforma fluida y reactiva, que funcionará como la **ventana del ciudadano al sistema de ciudad inteligente**.
- **Firmware y Diseño:**
 - **Arduino IDE:** Se empleará para refinar el *firmware* del microcontrolador, implementando la lógica avanzada para las alarmas diferenciadas.
 - **Software de Modelado 3D:** Se utilizará para el diseño preciso y funcional de la pieza a imprimir.

Descripción Detallada del Funcionamiento

El funcionamiento del sistema en esta fase se concibe como un flujo de datos continuo y orquestado, desde la captura física hasta la visualización en la palma del usuario:

1. **Captura y Publicación de Datos (Arduino -> Raspberry Pi -> MQTT):**
El Arduino lee constantemente sus sensores. Al detectar un evento (ej. un vehículo pasando con semáforo en rojo, un nivel de gas anómalo, un botón de pánico presionado), envía una trama de datos estructurada por el puerto serial. En la Raspberry Pi, un script de servicio escucha permanentemente este puerto, recibe los datos, los procesa y los publica de inmediato en el *broker* MQTT bajo un tópico específico y semántico (ej. `alertas/trafico/infraccion` o `sensores/ambiente/incendio/zona_historica_1`).
2. **Persistencia y Almacenamiento (MQTT -> API -> Base de Datos):**
La API del backend, a su vez, tiene un componente suscrito a los tópicos relevantes del *broker* MQTT. Al recibir un mensaje, la API lo valida, le da formato y lo inserta en la base de datos centralizada. Este paso es crucial, pues transforma un evento efímero en un **registro histórico persistente**, creando un repositorio de datos valioso para análisis futuros.

Datos que se procesaran en MQTT:

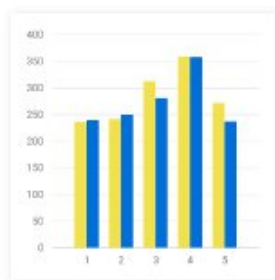
BUS1	Distancia de paradas, hora
BUS2	Distancia de paradas, hora
INFRACCION	Hora
PANICO	Hora
SISMO	Hora
GAS	Hora
ETC	...

3. Consumo y Visualización de Datos (API -> Aplicación Flutter):

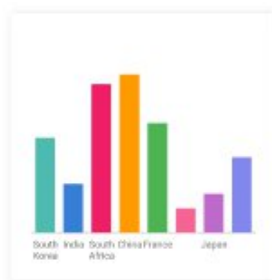
La aplicación móvil del usuario final se comunica exclusivamente con la API REST. Periódicamente o en respuesta a acciones del usuario, solicita información a través de *endpoints* específicos (ej. `GET /api/alertas/recientes` o `GET /api/transporte/ruta/1`). La API consulta la base de datos, formatea la respuesta (generalmente en JSON) y la devuelve a la aplicación, que se encarga de renderizarla de forma intuitiva en sus diferentes vistas:

- **Panel de Control:** Muestra gráficos y resúmenes de datos históricos filtrables (información almacenada en la base de datos).
- **Feed de Notificaciones:** Presenta las últimas alertas de seguridad en tiempo real (semáforo, pánico, etc.).
- **Mapa Interactivo:** Actualiza la posición simulada de los autobuses, ofreciendo una experiencia dinámica al usuario (mostrar distancia y tiempo estimado de llegada a la siguiente parada).

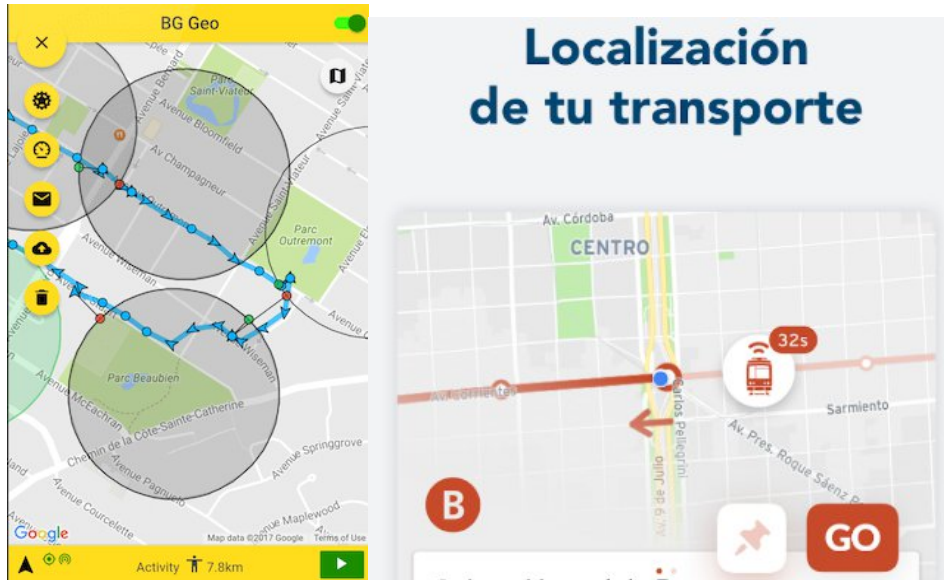
Ejemplos:



Numeric Axis



Category Axis



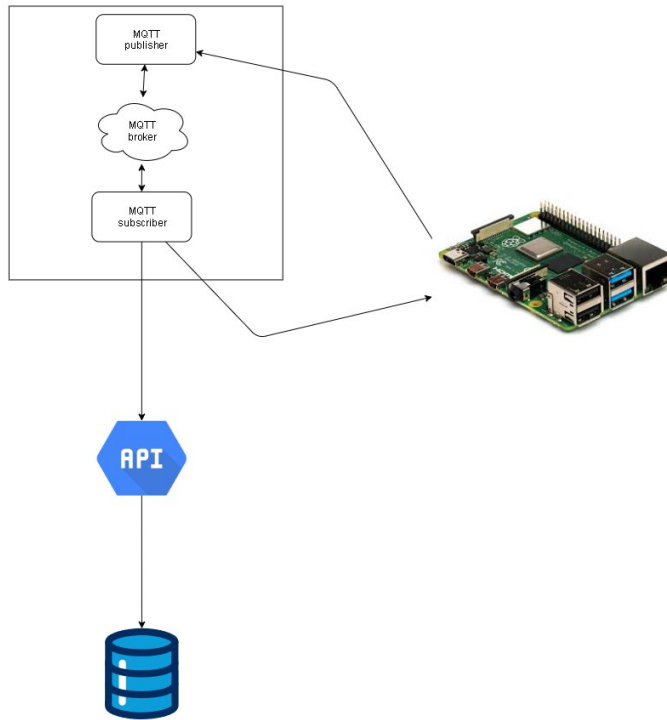
4. **Retroalimentación Local y Mejora Física:** Paralelamente al flujo de datos digital, el sistema mejora la respuesta local. El *firmware* actualizado del Arduino le permite activar **alarmas acústicas diferenciadas**, enriqueciendo la semántica de las alertas a nivel físico. Finalmente, la **pieza impresa en 3D se integra en la maqueta**, demostrando no solo una mejora estética, sino la capacidad de adaptar y evolucionar el hardware a las necesidades específicas del entorno simulado.

Las piezas 3d pueden ser impresas en fat lab sus requisitos son:

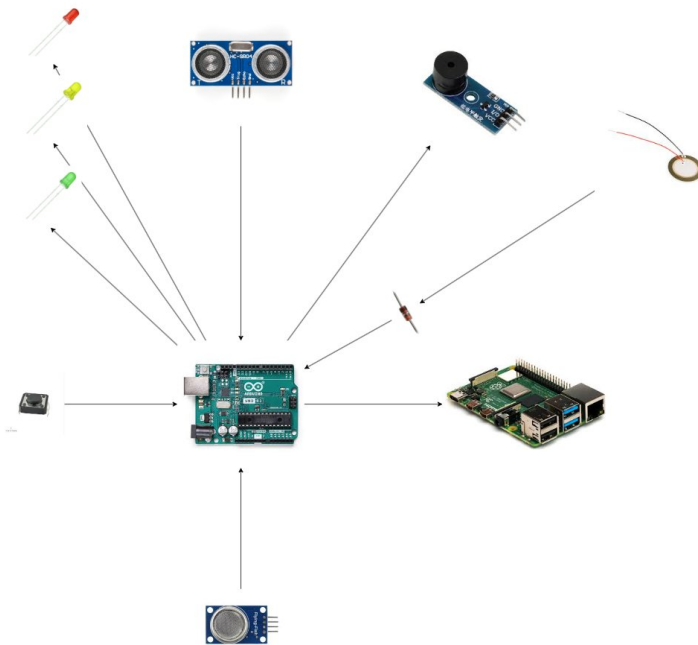
- Llevar su propio filamento, evitar que agarre humedad el filamento.
 - En dado caso tiene humedad llevarlo un día antes para ser procesado el filamento.
- Llevar la [carta firmada](#)
- Formato del archivo para ser impreso stl

Diagramas Fase 2

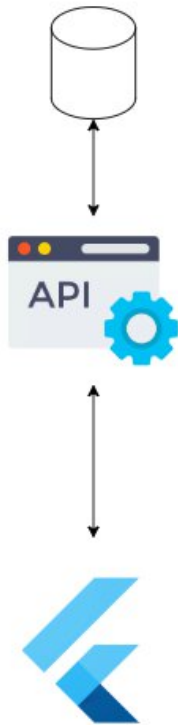
MQTT



RASPBERRY Y ARDUINO



BASE DE DATOS Y FLUTTER



NOTAS:

Las funcionalidades anteriores de la fase 1, se siguen manteniendo y processing ya no será el medio de mostrar la información si no flutter.

Pueden Seguir mejorando y agregando a la maqueta para mejorar la presentación de esta fase.

4.3 Entregables

Tipo	Descripción
Prototipo físico	Se realizará la documentación correspondiente de la pieza 3d elegida a su elección.
Maqueta del prototipo propuesto	Importante recordar que el enfoque general es sobre un Sistema inteligente de Monitoreo de seguridad mediante IoT, por lo cual se necesita una maqueta física que encapsule los componentes utilizados en la fase y se oriente hacia el entorno del sistema.

Modelo 3d Impreso	Modelo 3d impreso a su elección incluida en su maqueta.
Smart connected desing framework	Descripción de las capas de Smart Connected design Framework utilizadas en la fase.
Diagramas	Diagramas a criterios del estudiante que ayuden a evidenciar el flujo de la información.
Documentacion semanal de los aportes.	Se realiara la documentacion de lo que cada integrante aporte en el poyecto.

5. Metodología

Para la ejecución de la Fase 2, se seguirá una metodología incremental y estructurada, enfocada en la integración de componentes de software y hardware para construir una red IoT funcional. El proceso se divide en tres etapas principales:

1. Investigación y Planificación Arquitectónica:

- **Definición de la Pila Tecnológica (Tech Stack):** Se investigarán y seleccionarán las tecnologías específicas para el backend. Esto incluye la elección de un *broker* MQTT (ej. Mosquitto), la selección del sistema gestor de base de datos (SQL como PostgreSQL vs. NoSQL como MongoDB) basándose en los requerimientos de consulta y la naturaleza de los datos, y la elección de un framework para el desarrollo de la API (ej. Flask/FastAPI en Python, Express en Node.js).
- **Estudio de Protocolos y Comunicación:** Se profundizará en el funcionamiento del protocolo MQTT (tópicos, QoS, publicadores/suscriptores) y en el diseño de APIs RESTful para asegurar una comunicación eficiente y estandarizada entre el servidor y la aplicación móvil.
- **Planificación de la Arquitectura de Datos:** Se planificará la estructura de la base de datos (esquemas, tablas o colecciones) y se definirá el "contrato" de la API, especificando los *endpoints*, los métodos HTTP y los formatos de datos (JSON) para cada recurso.

2. Diseño y Prototipado:

- **Diagramas de Arquitectura y Flujo de Datos:** Se actualizarán y crearán nuevos diagramas que reflejen la arquitectura completa de la Fase 2. Esto incluirá el flujo de datos desde el Arduino, pasando por la conexión serial a la Raspberry Pi, la publicación en el *broker* MQTT, la persistencia en la base de datos a través de la API, y finalmente el consumo de datos por la aplicación Flutter.

- **Diseño de la Interfaz de Usuario (UI/UX):** Se realizarán bocetos y *wireframes* de las diferentes pantallas de la aplicación móvil en Flutter. Se diseñará la disposición del dashboard de históricos, la sección de notificaciones y la vista del mapa interactivo para asegurar una experiencia de usuario clara e intuitiva.
- **Diseño del Modelo 3D:** Se utilizará software de modelado 3D (como Tinkercad, Fusion 360, etc.) para diseñar la pieza ergonómica que se integrará en la maqueta física, asegurando que sus dimensiones y estética complementen el prototipo o pueden buscar un modelo ya pre echo en internet.

3. Implementación e Integración:

- **Desarrollo del Backend:**
 - Configuración de la Raspberry Pi como nodo central y servidor.
 - Instalación y configuración del *broker* MQTT.
 - Desarrollo de los *scripts* que leen los datos del puerto serial, los procesan y los publican en los tópicos MQTT correspondientes.
 - Implementación de la API REST y la lógica para suscribirse a los tópicos MQTT y almacenar los datos en la base de datos seleccionada.
- **Desarrollo del Frontend:**
 - Creación del proyecto en Flutter.
 - Implementación de las vistas (widgets) diseñadas en la fase de prototipado.
 - Integración de la aplicación con la API REST para obtener y visualizar los datos de los sensores, las alertas y la ubicación simulada de los autobuses.
- **Integración de Hardware:**
 - Refinamiento del código en el Arduino para implementar el sistema de alarmas diferenciadas.
 - Impresión de la pieza 3D y su montaje físico en la maqueta.

6. Desarrollo de Habilidades Blandas

Para complementar el desarrollo técnico, esta sección se centra en las habilidades blandas que los estudiantes deberán mejorar a lo largo del proyecto, como la comunicación, el liderazgo, y la colaboración en equipo.

6.1 Proyectos en Grupo

Este proyecto se desarrollará en equipos, fomentando la colaboración y distribución de responsabilidades bajo un modelo flexible, adaptado a las necesidades del curso.

Estructura sugerida:

- **Roles no fijos:** Los integrantes rotarán funciones (coordinación, desarrollo, documentación) para garantizar participación equitativa.
- **Autogestión:** Cada equipo definirá su método de trabajo (reuniones semanales, división de tareas).

6.1.1 Herramientas de Organización

- **GitHub:** Control de versiones para código y documentación.
- **Plataformas colaborativas:** Google Drive o Trello para asignar tareas y compartir avances (opcional).

6.1.2 Comunicación y Retroalimentación

- **Revisiones periódicas:** Presentaciones breves en clase para mostrar progresos y recibir feedback del profesor.
- **Resolución de conflictos:** Los equipos deberán resolver discrepancias internamente, con apoyo del docente si es necesario.

7. Cronograma

FASE 1

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	sábado 23 de Agosto	sábado 23 de Agosto
Elaboración	sábado 23 de Agosto	viernes 12 de Septiembre
Calificación	sábado 13 de Septiembre	sábado 13 de Septiembre

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Si/No)
Cumplimiento de la tecnología establecida	Uso de arduino, raspberri, flutter y componentes eléctricos.	
Entrega de la fase	Entregar la Fase 1.	

Prototipo Físico	Se debe entregar el prototipo físico en funcionamiento con su respectiva pieza en 3d impresa incluida en la maqueta.	
Documentación obligatoria	Se debe entregar en la documentación del prototipo, los semanales, los diagramas que crea correspondiente.	
Entregables	Se deberán de enviar todos los entregables de la documentación.	
Github	Todo el código utilizado y la documentación deberá ser subido a un repositorio de github.	
Nombre del repositorio	ARQUI2B_2S2025_G<INICIAL><NUMERO>	
Usuarios de Github	Agregar el usuario del auxiliar como colaborador a su repositorio de github	
Nombre Carpeta de la fase	Debido a que se usará el mismo repositorio durante todo el semestre, se solicita que contenga 3 carpetas, en las cuales se presenta en cada fase.	

- Ejemplo:
 - Inicial A o L
 - Grupo 1-4
 - ARQUI2B_2S2025_GA1
- Github
 - Janjo25
 - AlexCB-16
- Nombre de la Carpeta
 - FASE 1
 - FASE 2
 - FASE 3
- Calificación será grupal y otra parte individual

8.2 Valores

En el desarrollo del proyecto, se espera que cada estudiante demuestre honestidad académica y profesionalismo. Por lo tanto, se establecen los siguientes principios:

1. **Originalidad del Trabajo**
 - Cada estudiante o equipo debe desarrollar su propio código y/o documentación, aplicando los conocimientos adquiridos en el curso.
2. **Prohibición de Copias y Plagio**
 - Si se detecta la copia total o parcial del código, documentación o cualquier otro entregable, la calificación será de **0 puntos**.

- Esto incluye la reproducción de código entre compañeros, la reutilización de proyectos de semestres anteriores o el uso de código externo sin la debida referencia.
- 3. **Uso Responsable de Recursos Externos**
 - El uso de bibliotecas, frameworks y ejemplos de código externos está permitido, siempre y cuando se referencian correctamente y se comprendan plenamente. (Consultar con el catedrático su política)
- 4. **Revisión y Detección de Plagio**
 - Se podrán utilizar herramientas automatizadas y revisiones manuales para identificar similitudes en los proyectos.
 - En caso de sospecha, el estudiante deberá justificar su código y demostrar su desarrollo individual o en equipo. Si este extremo no es comprobable la calificación será de **0 puntos**.

Al detectarse estos aspectos se informará al catedrático del curso quien realizará las acciones que considere oportunas.

8.3 Comentarios Generales

- Todas las aclaraciones se realizarán en clase, por lo que deberán acatar todas las instrucciones escritas y verbales al momento de explicar el proyecto.
- Se calificará solamente lo que sea completamente funcional.
- Se calificará los commits individualmente.